Combat ID and Threat Prioritization with Real Trackers

Part 2: replacing association by label with multi-target trackers in cidgen

Draft, 11 November 2026. Companion to Part 1 (CID_Tier3_Report, 8 November 2026). All platform, sensor and doctrine values are notional.

1 About this part

Part 1 described the study system and its results with one important stand-in: every radar report arrived tagged with the object that produced it, clutter was removed before tracking, and one filter ran per true object ("association by label"). Part 1 ranked that stand-in as the furthest from a fielded system and put a real association and tracking stage first among its next steps (Part 1, Tables 14 and 22).

This part reports that step. Five trackers were inserted into the chain in front of the unchanged feature, Combat ID (CID) and threat prioritization (TP) stages, and the whole chain was re-run on S12. The focus here is system-level: what changes downstream when the tracker is real, where the losses come from, which tracker choices matter, and what they cost to run. How the trackers work inside, and how they differ algorithmically, is left to a separate technical note.

Three points apply throughout:

- **Same worlds, different tracking.** A tracker scenario (for example S12_gnn) uses the same seed, objects and sensor reports as S12. Only the tracking differs, so label-mode and tracker runs of a seed compare directly.
- **Unless stated, results are on the S12 test seeds 18–20 with perfect identity and the doctrine ranker,** so that tracking effects are not mixed with identification errors. Section 5.2 adds the CID models.
- **Truth is used only to score.** The trackers never see the true object id; truth is matched to tracks afterwards to build labels and score results, as before.

Section 2 describes what changed in the chain, Section 3 the trackers, and Section 4 a change to how the threat list is scored, which the results depend on. Sections 5–8 give the results, Section 9 the run-time cost, Section 10 the Stone Soup MFA integration, and Sections 11–12 revisit the stand-ins and conclude.

Contents

2 What changed in the chain

Figure 1 shows the change. In Part 1 the association box was a stand-in that read the truth. In Part 2 a tracker receives every radar report, clutter included, with the object id hidden, and builds tracks the way a fielded tracker would: gate reports against predicted tracks, assign them, update, start tentative tracks from unassigned reports, confirm them after enough hits, coast and delete them, and drop duplicates. Only confirmed tracks go on to the feature grid.

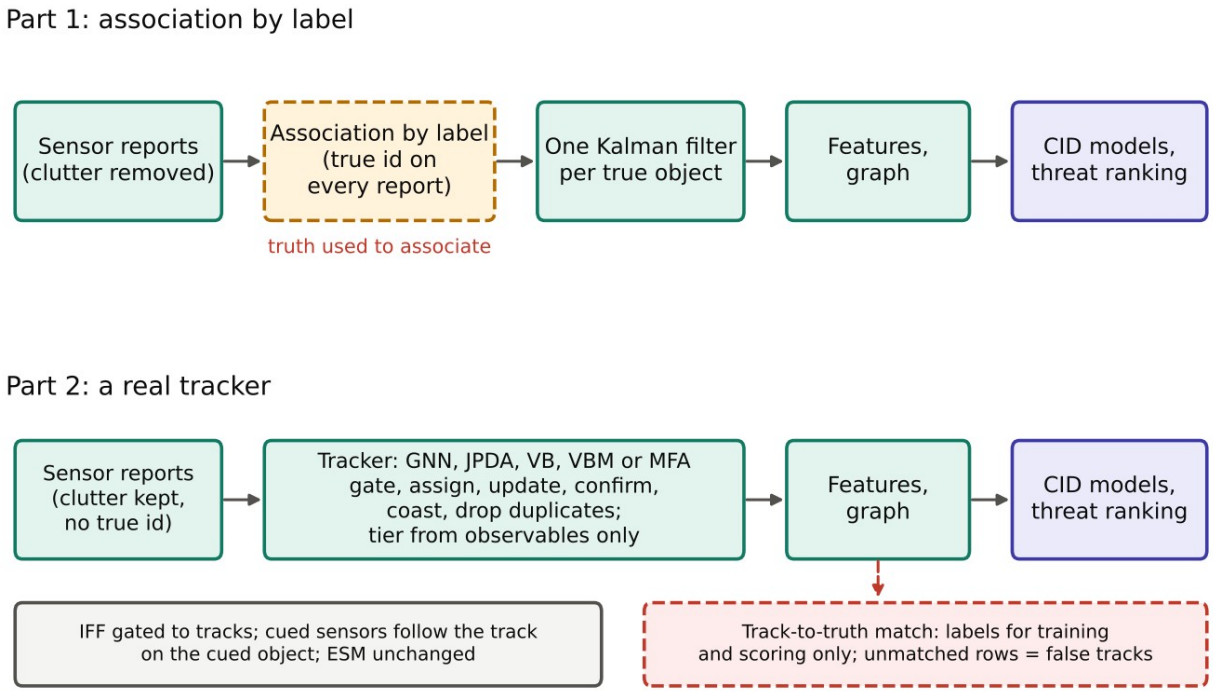


Figure 1. The chain in Part 1 and in Part 2. Teal boxes are working algorithms, the dashed amber box was a stand-in, and the dashed red box marks where truth is used.

Several other parts of the chain had to change with it:

- **Tracker settings come from observables.** Each track's filter settings (process noise, initial velocity uncertainty, coast time) depend on its tier. The tracker guesses the tier from the first report (altitude of at least 15 km or radial speed of at least 1,000 m/s means Tier 3; the counter-UAS radar, or low and small, means Tier 1) and fixes it after a 10 s acquisition window by the observable rule of Part 1. Part 1's label mode chose these settings from the true tier.
- **Radar registration bias is allowed for.** Each radar has a fixed registration bias. The tracker knows only its specified size and adds it to every report's noise. Without this, two radars see one aircraft about one sigma apart and duplicate tracks result: on one S12 run, adding it cut false-track rows from 13.9% to 3.4%.
- **The other sensors reach tracks through association too** (Table 1).
- **Tracks are matched to truth for labels and scoring only.** At every grid time, confirmed tracks are matched one-to-one to true objects (Hungarian assignment, gate max(2 km, 4 track sigmas), with preference for keeping the previous pairing). A matched track-snapshot takes the object's labels. An unmatched one is a **false-track row**: it stays in the track graph as a neighbor but, like birds, is not scored by CID or TP.

Table 1. How each sensor's reports reach tracks.

Sensor	Association by label (Part 1)	Real tracker (Part 2)
Radar	The report's true object id	The track the tracker assigned the report to; clutter reports reach the tracker
IFF	The reply's id (garbled with a small probability)	Gated to the nearest track by measured range and azimuth (2-D chi-square 9.21)
JEM, LIDAR, EO/IR (cued)	The id the cue went to	The track that follows the object the cue went to; the generator's cue errors are kept
ESM	Linked to tracks by a geometric gate	Unchanged

Label mode is still the default and reproduces Part 1 byte for byte (checked on S12 seed 15), so every Part 1 result stands. Each tracker run also writes new tables: the track-to-truth match, one row per track, one row per detected object (coverage, tracks used, id switches, confirmation delay, error) and a run summary.

3 The trackers

All five trackers share cidgen's track management: tentative tracks from unassigned reports, confirmation at 3 hits within 10 s, deletion after the tier's coast time, duplicate dropping and the observable tier. They differ only in how a confirmed track uses each radar look (Table 2). This keeps the comparison about association rather than about bookkeeping. One setting added during the study, **protection of resolved tracks** (Section 8.3), is available to every tracker.

Table 2. The trackers compared. The algorithms are described in a separate technical note.

Method	How a confirmed track uses one radar look	What it is meant to handle	Source
gnn	Global nearest neighbor: one-to-one hard assignment at least total cost (chi-square gate 16)	The baseline; fast	Written for cidgen
jpda	Joint probabilistic data association: exact probabilities of every report-to-track assignment, then a mixture update	Crossing and closely spaced targets	Written for cidgen
vb	Variational Bayes: the same probabilities, a soft update, and every 2 s a re-solve of the last 20 s with a smoother	Closely spaced targets without JPDA's coalescence	From the team's VB tracker work (Turner, Bottone and Avasarala, NIPS 2014)
vbm	VB, plus the option that one report stands for several tracks inside one radar resolution cell	Groups the radar cannot resolve	Written for cidgen
mfa	Multi-frame assignment over the last 3 looks, solved as an integer program	Hard association decided with a short look-back	Stone Soup 1.9.1 (Dstl), adapted

Every tracker is selected with one line in the scenario file (tracking.method), and ready-made scenarios exist for each: S12_gnn, S12_jpda, S12_vb, S12_vbm and S12_mfa, the protected variants S12_gnnp and S12_vbp, and S12F_gnn and S12D_gnn for the Part 1 robustness tests. Each tracker can also be timed section by section with --debug 1 (Section 9).

4 Scoring the threat list against all threats

Part 1's threat-list measures were computed over the tracks in the picture. With association by label that was harmless: every object detected had exactly one track. With a real tracker it flatters the result. A threat the tracker has not yet confirmed, has dropped, or holds only inside a group track is

simply absent from the picture and is never scored as a miss. Track breaks also multiply the number of "threat episodes" scored: 298 with the GNN against 126 in label mode on the same three runs.

The old measure shows the problem plainly. With perfect identity, the GNN's tracked-only top-3 capture was 89.8%, higher than label mode's 89.1%, although the GNN holds fewer threats. The TP evaluation was therefore extended to score against every hostile object's true threat, recomputed from the truth states with the same formula the labels use, from the object's first radar report onward (Table 3). The original measures are still produced, so Part 1 results stay comparable, and label-mode runs get the new measures too as the baseline.

Table 3. Threat-list measures scored against all threats.

Measure	Meaning
Own track	Share of real-threat object-snapshots whose object has its own track in the picture
Covered	… or lies within 500 m of some track (for example, an unresolved group member sharing a track)
Captured in top 3 (all threats)	Threat held by the top-3 tracks divided by the best any 3 objects could hold, counting every hostile object from its first radar report, tracked or not; a group track earns one member's threat
Never in top 3 (all threats)	Share of threatening objects never covered by a top-3 track

5 Results with the GNN tracker

The global nearest neighbor (GNN) tracker was run on all 20 S12 seeds, and the full chain was retrained and retested on its output with the usual split (training seeds 1–17, test seeds 18–20).

5.1 Tracking quality

Table 4 summarizes the tracker against truth. Most objects are covered nearly all the time once detected; the means are pulled down by a few hard objects. Across the 20 runs, an average of 4.1% of track-snapshots are false-track rows (2.4–7.3% by run), and about 230 duplicate tracks are dropped per run.

Table 4. GNN tracking quality on 20 S12 runs.

GNN tracker, 20 S12 runs	Tier 1	Tier 2	Tier 3
Objects detected (all runs)	240	875	340
Coverage per object: median (mean)	0.996 (0.85)	0.995 (0.94)	0.897 (0.88)
Tracks per object: median (90th percentile)	1 (4)	2 (18)	2 (3)
Confirmation delay, median	1 s	2 s	8 s
Position error, median	37 m	267 m	236 m
Label mode, for comparison	Coverage 1.00, one track per object, no false tracks		

- **Tier 3: the loss is at track start.** At long range the sector radar sees a Tier 3 object only every 5–7 s, so collecting 3 hits takes 10–20 s, and a short coast can drop a young track in a gap. On the test seeds, mean Tier 3 coverage is 0.90 for every tracker tested (Section 6), which shows the limit is detection rate and confirmation, not association.
- **Tier 2: tight formations break tracks.** The 90th percentile of 18 tracks per object comes from formations whose members are 200 m–1 km apart, about the radar's cross-range error at 50 km. Reports merge and split, and tracks break and swap between members.
- **Tier 1: one swarm.** Figure 2 shows Tier 1 coverage by run. In the runs checked (seeds 18 and 20), the shortfall is the hostile drone swarm alone; own and hobbyist drones are covered 100% of the time. Section 5.4 explains why.

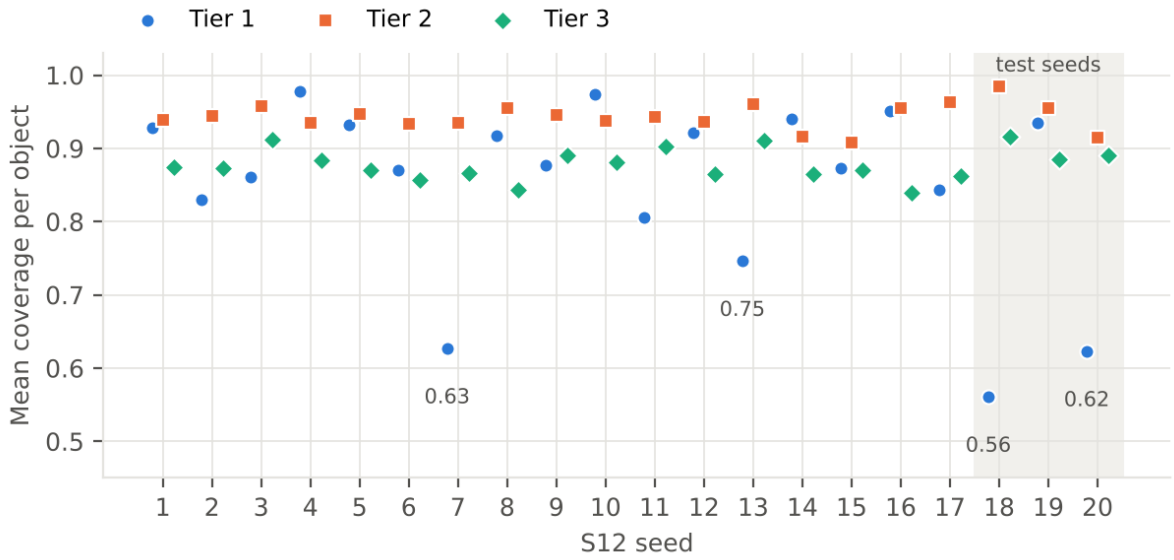


Figure 2. GNN tracker: mean coverage per object, by tier, for each of the 20 S12 runs. Labels mark the four runs with Tier 1 coverage below 0.8.

5.2 Combat ID on real tracks

Identification barely moves (Table 5). Behind the GNN, the tier router scores 0.970, 0.992 and 0.998 balanced accuracy on Tiers 1, 2 and 3, against 0.978, 0.997 and 1.000 in label mode. The gradient-boosted trees remain the strongest single model. As in Part 1's degraded-sensing test, the neural models' remaining errors are own drones: the graph network called 4.3% of own-drone snapshots hostile, all of them when the EO/IR camera had not seen the drone.

Table 5. Combat ID behind the GNN tracker (S12 test seeds 18–20; 36 Tier 1, 126 Tier 2 and 51 Tier 3 objects; matched track-snapshots only).

Model (one training seed)	Tier 1 bal. acc.	Tier 2 bal. acc.	Tier 3 bal. acc.	Tier 1 friend → hostile	Tier 2 missed hostile
XGB-own	0.996	1.000	1.000	0.3%	0.05%
GNN-EG (graph network)	0.961	0.993	1.000	4.3%	1.8%
GRU	0.979	0.992	0.998	3.0%	1.3%
Tier router	0.970	0.992	0.998	4.3%	1.3%
Tier router, label mode (Part 1, three seeds)	0.978	0.997	1.000	–	0.7%
BN-learned	0.975	0.958	0.985	0.7%	1.2%

Neural models are from one training seed; Part 1 showed single seeds can differ by about 0.01, so differences under that size should not be read as real. CID is scored on matched track-snapshots only; false-track rows are not scored (Section 11).

5.3 Threat prioritization

The real tracker costs about 4 points of top-3 capture (Table 6). With perfect identity, the share of real threat captured in the top 3 falls from 88.9% to 85.2%, and the share of threats with their own track falls 12 points. Almost every threat is still covered by some track (97.8%), and the share of threatening objects that never reach the top 3 is unchanged.

The CID stack stays at the ceiling. Behind the GNN, the stack captures 85.3% against 85.2% with perfect identity, and places no friends in ENGAGE. The loss is entirely the tracker's, not the identification models'.

Table 6. Threat prioritization behind the GNN tracker, scored against all threats.

Measure (seeds 18–20, doctrine ranker)	Label mode	GNN
Real threats with their own track	97.3%	85.4%
Real threats covered by some track	99.3%	97.8%
Captured in top 3, all threats: perfect identity	88.9%	85.2%
Captured in top 3, all threats: stack	88.7%	85.3%
Threatening objects never in the top 3 (perfect identity)	7.1%	7.1%
Friendly track-snapshots in ENGAGE (stack)	0.06%	0%

5.4 Where the loss comes from: groups the radar cannot resolve

Almost all of the gap between label mode and the GNN comes from **unresolved groups**:

- **The hostile drone swarm.** Eight drones fly about 55 m apart, 13 m at the closest. The radars merge them into one or two reports per look. Every member stays within 200 m of a track, but only 34–42% have their own track. Label mode held 82% of the swarm only because each merged report updated whichever member returned the strongest echo, a member that changes from look to look.
- **Tier 2 escort and helicopter formations** show the same effect, less severely.

This is a sensor-resolution limit, not a GNN tuning fault: no association method can give a member its own track if no report ever stands for that member alone. Sections 6–8 test how far better association, and better handling of merged reports, can recover it.

6 Do soft-association trackers help on S12?

JPDA and VB replace the GNN's hard choice with assignment probabilities. On their own, they did not help on S12 (Table 7, Figure 3). Coverage is the same to within a point in every tier, the threat measures are within 1.5 points of the GNN, and both trackers churned through far more tracks: JPDA confirmed 2.3 times as many tracks as the GNN and dropped 6.6 times as many duplicates.

Table 7. GNN, JPDA and VB on the S12 test seeds (VB with 5 iterations; no protection).

Seeds 18–20, mean per run	GNN	JPDA	VB
Confirmed tracks	897	2,098	1,586
Duplicate tracks dropped	228	1,497	936
Coverage per object, mean (Tier 1 / 2 / 3)	0.71 / 0.95 / 0.90	0.69 / 0.95 / 0.90	0.70 / 0.95 / 0.90
Tracks per object, mean	7.6	8.0	7.8
False-track rows	4.2%	4.5%	4.5%
Position error, median	86 m	79 m	86 m
Real threats with own track	85.4%	83.9%	85.0%
Real threats covered	97.8%	97.8%	97.7%
Captured in top 3, all threats (perfect identity)	85.2%	84.1%	84.4%

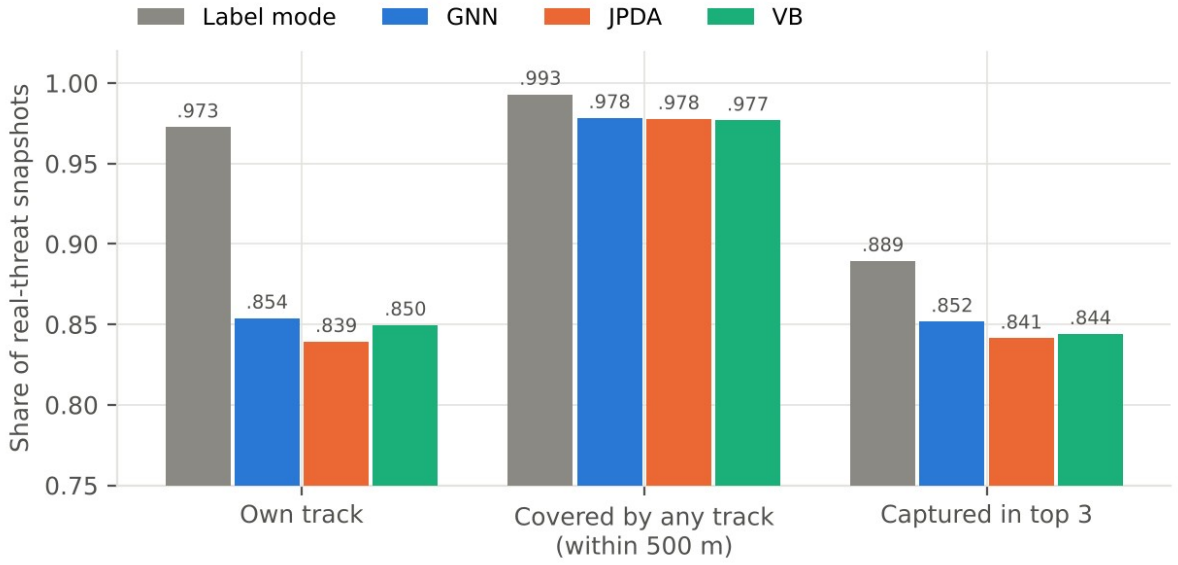


Figure 3. Threat measures on the S12 test seeds (perfect identity, doctrine ranker). Label mode is Part 1's stand-in.

Two things explain the null result. First, the swarm is the main loss, and no one-report-per-object tracker can recover members the radar never resolves. Second, as Section 8 shows, the soft trackers did hold more swarm members apart, but duplicate dropping then deleted them as duplicates. This was only found with the bench.

7 The tracker bench: testing where the answer is known

Because the VB tracker is new as a tracker in its own right, and the S12 result could have hidden a fault, each tracker was first tested on simple cases with a known answer (tracker_bench.py). One radar (100 m noise per axis, a look every 2 s, detection probability 0.9, clutter at the density the trackers assume) watches two aircraft crossing at 10°, 30° or 90°, or four aircraft flying line abreast at 1 to 8 radar sigmas apart before fanning out. Each tracker is compared with an oracle: the same filter fed the true associations. Twenty Monte Carlo runs per case.

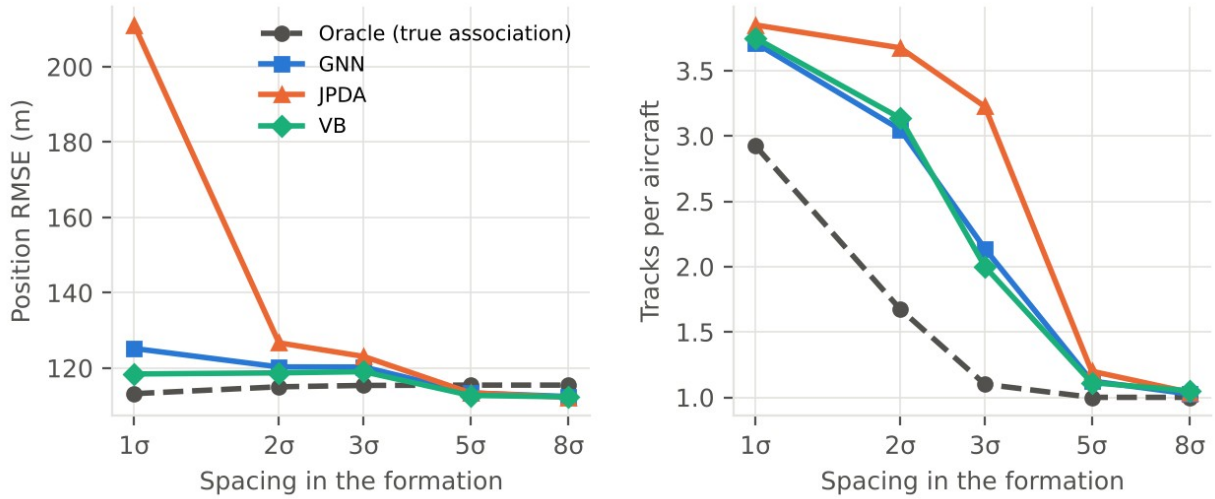


Figure 4. Four-aircraft formation on the bench: position error and tracks per aircraft against the spacing between aircraft (20 runs each).

- **When targets are well separated** (crossings at 30° and 90°, formations 5σ or more apart), every tracker matches the oracle. The extra machinery in JPDA and VB does no harm when association is easy.
- **In close formations JPDA coalesces.** At 1σ its tracks are pulled together (error 211 m against 113 m for the oracle), and its uncertainty is overstated (normalized estimation error 1.7, where 3 is consistent). This is JPDA's known weakness, and it is why JPDA was not carried further.
- **VB does not coalesce.** Its window re-solves the assignments with smoothed states. It has the lowest error of the three real trackers at every spacing (118–119 m at 1σ–3σ), slightly fewer breaks than the GNN at 3σ, and consistent uncertainty.
- **Below 3σ, no tracker approaches the oracle.** That gap is what association by label used to hide.

The bench passed the project team's test for continuing with VB: it does not fall short of JPDA, it beats it where JPDA is weakest, and it is consistent.

8 Groups the radar cannot resolve

Two approaches to unresolved groups were tried. Option B leaves the tracker alone and estimates how many objects each track stands for. Option A changes the tracker so that one report may stand for several tracks.

8.1 Counting group members from RCS and nearby echoes (option B)

The idea, from the project team, is that a group's summed radar return behaves differently from one aircraft's. The estimator uses two cues from each track's last 20 s of reports: how much the radar cross-section (RCS) fluctuates (the summed echo of several independently fluctuating members is steadier, and the measure needs no aircraft type or aspect), and how many other echoes fall within 500 m in the same look (a partly resolved group throws extra echoes around its track). A calibration table fitted on training runs maps the two cues to an expected member count.

Table 8. How well each cue separates groups from single aircraft (GNN; calibrated on seeds 15–17, tested on 18–20).

Cue (AUC; 0.5 = chance, 1 = perfect)	Group of 2 or more	3 or more	5 or more
RCS fluctuation alone	0.57	0.58	0.60
Nearby echoes alone	0.83	0.80	0.73
Both (the estimator)	0.86	0.85	0.81

The estimator separates groups well, mostly through the nearby echoes; RCS fluctuation adds most for large groups. But large groups are undercounted: groups of 5 or more average 2.7 estimated members.

Weighting the threat by members does not pay (Table 9). Even with true member counts, it moves group tracks above dangerous single aircraft: the group's threat mass in the top 3 rises 4 points while per-object capture falls 2.6. With the estimated counts, both measures fall. Whether eight small drones outrank one strike aircraft is a doctrine decision, not something the estimator can settle. The estimate is better used as information: a CID feature, or a "group, about n" tag on the operator's track.

Table 9. Threat list with and without member weighting (S12 test seeds, doctrine ranker).

Threat weighted by members (GNN, perfect identity)	Per-object top-3 capture	Group-credited capture	Threat mass in the top 3
None (as before)	85.1%	89.4%	80.9%
Estimated members	84.2%	88.4%	78.6%
True members (upper bound)	82.5%	88.0%	85.1%

8.2 Merged reports in the tracker (option A, VBM)

VBM is the VB tracker with one change: a report may be explained by a set of tracks, if every pair could lie inside one resolution cell of that radar. Each member of the set is told where the others are, which holds the members apart. Merged reports, a counter-UAS radar and drones on slow loops were added to the bench to test it.

On that swarm bench VBM gave the best results of the first comparison: for 8 drones on 25 m loops, 63% of drone-time had its own track, against 55% for the GNN, with fewer tracks per drone over its life (5.3 against 6.6).

8.3 Ablation: protecting resolved tracks is what matters

VBM made two changes at once: the merged-report sets, and a rule that two tracks which took different reports in the same look within the last 30 s are two objects and are never dropped as duplicates (**protection of resolved tracks**). Switching each off in turn showed that **nearly all of the gain came from protection** (8 drones: 0.647 with both, 0.652 without the sets, 0.538 without protection). Protection was then made a setting for every tracker.

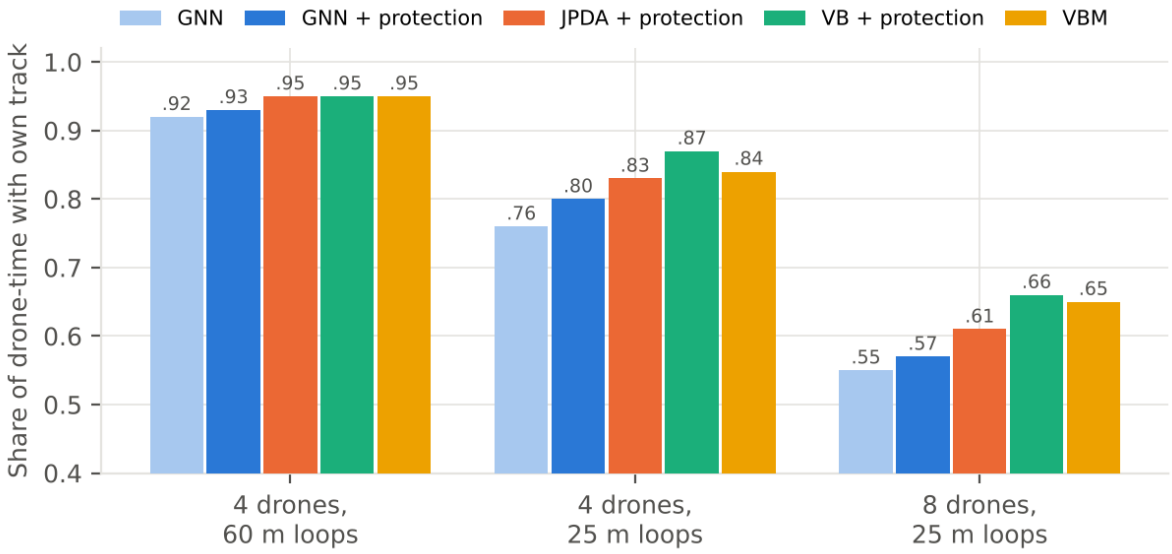


Figure 5. Swarm bench: share of drone-time with the drone's own track, with protection on for every tracker except the plain GNN (4 runs of 180 s each).

With protection on, soft association does help in swarms (Figure 5): VB gains about 10 points over the plain GNN and JPDA about 6. Without protection, duplicate dropping had been deleting exactly the members VB was holding apart, which is why JPDA and VB showed nothing on S12 in Section 6. The merged-report sets add nothing beyond VB with protection. They also cannot start a track for a member the radar never resolves, so the 8-drone swarm still holds only about 5 tracks for 8 drones.

8.4 Protection on S12

Table 10. Protection of resolved tracks on S12 seed 20 (perfect identity, doctrine ranker).

S12 seed 20 (perfect identity)	Own track	Covered	Top-3 capture	Tracks per object	Duplicates dropped
Label mode	96.8%	99.8%	88.7%	1	0
GNN	78.5%	97.6%	80.2%	5.8	243
GNN + protection	82.6%	98.0%	80.5%	5.3	–
VB	78.2%	97.0%	80.4%	6.1	982
VB + protection	83.1%	97.9%	80.3%	4.9	44

- **Protection is the one change worth keeping.** It costs nothing, works with every tracker, and adds 4–5 points of own-track share on seed 20. Over seeds 18–20, the GNN with protection reaches 87.2% own track and 85.9% top-3 capture, against 85.4% and 85.2% without it.

- **VB with protection holds tracks together best** (4.9 tracks per object, very few duplicates dropped), but beats the GNN with protection by only 0.5 points of own-track share.
- **Top-3 capture does not move** with any tracker on this seed (80.2–80.5%). The threats that drive the top 3 are mostly already tracked; the members recovered are mostly swarm drones, which rank below the top 3.

9 Run time

A real tracker must keep up with the radar. To see where time goes, every tracker can be run with --debug 1, which times each section of the code (gating, clustering, assignment probabilities, updates, the VB window) and writes a time-stamped log with one line per timed block, flushed every 5 s so a long run's progress is visible, plus a summary table. With the flag off, the timers do nothing and results are identical.

VB spends 95% of its time in the window step (S12 seed 20, first 120 s). Inside it, 57% goes to assignment probabilities, 24% to the smoother and 11% to scoring, spread over tens of thousands of small calls rather than any one expensive call. The per-look filter step is only 5%.

One iteration is enough (Figure 6). The window step repeats its assignment-and-smoothing pass until nothing moves by more than 1 m, up to 5 times. Allowing only one pass gave the same coverage (0.841 against 0.841), almost the same tracks per object (1.82 against 1.78), the same swarm coverage, and slightly lower position error, in 3.7 times less time. The same held on the swarm bench. S12_vbp now uses one iteration.

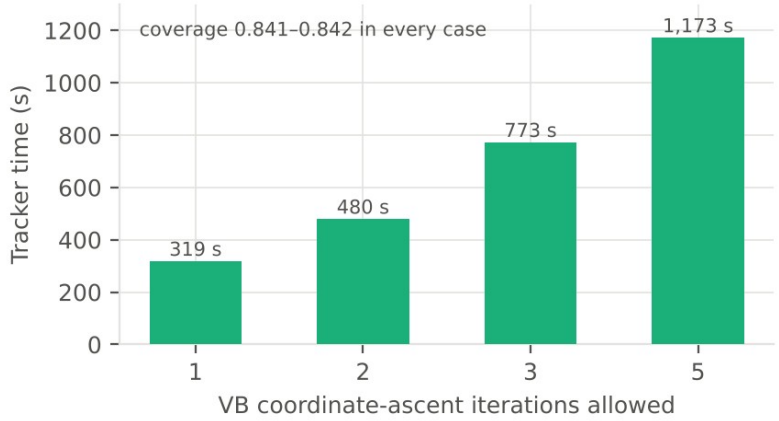


Figure 6. VB with protection, S12 seed 20, first 900 s: tracker time against the number of iterations allowed.

Table 11. Tracker run time.

Tracker (one full S12 run, about 1,800 s simulated)	Tracker run time	Where
Label mode, GNN, GNN + protection	About 1 min	Either machine
VB (5 iterations)	About 20 min	Cloud workspace
VB + protection (5 iterations)	45 min; 37 min	Cloud workspace; team workstation
VB + protection (1 iteration)	319 s for the first 900 s simulated	Cloud workspace
Stone Soup MFA + protection	92 s for the first 60 s simulated; full runs in progress	Team workstation

For the study, the GNN with protection is about 45 times cheaper than VB with protection at 5 iterations, and still roughly ten times cheaper at one iteration, for nearly the same tracking. VB's advantage is in close groups, and it would matter more in scenarios with more of them.

10 Stone Soup multi-frame assignment (MFA)

The fifth tracker uses an outside, published implementation: the multi-frame assignment data associator in Stone Soup 1.9.1, the open-source tracking framework from the UK's Dstl, with the OR-Tools integer solver. MFA keeps several association histories per track over the last few looks and picks the jointly best set, so a hard decision can be revised for a short time. It was integrated the same way as JPDA and VB: cidgen keeps its own track management and protection, and Stone Soup handles association and update for confirmed tracks, one call per cluster of tracks and reports in each radar look, with each report's own noise and each track's own process noise.

Before integration, a probe script checked the installation, MFA on two crossing targets, per-track process noise, and the speed on the team's workstation. Integration then exposed a series of problems, all in how the library meets a dense, 30-minute scenario rather than in the method (Table 12).

Table 12. Problems found integrating Stone Soup MFA, and the changes made.

Problem found	Cause	Change
"Cost matrix is infeasible" after a few looks	Capping each track's hypotheses could remove every hypothesis that avoids a conflict with another track	Always keep each track's "missed since the window start" hypothesis beside the top 15
Very slow: one large problem per look	All confirmed tracks and reports of a look went into one call	Split each look into clusters of tracks and reports linked by the gate, one small call each, with report numbering shared across the look so histories agree
Hypothesis explosion (one look took 32 s at 6 s simulated)	No cap on hypotheses per track	Keep the 15 most likely plus the all-missed hypothesis
119 tracks and 16.9% false-track rows in the first 60 s	Only the single best hypothesis's report was marked used, so other reports started new tracks; the track state was the best hypothesis only	Use the mixture of the kept hypotheses as the state; a report counts as used if the tracks explain it with total probability at least 0.5. Result: 81 tracks, 0% false-track rows
Crashes in full runs (infinite costs, "infeasible primal problem" in the integer solver, an index error in pruning)	Hypotheses with zero weight give infinite costs	Floor tiny weights; catch solver failures, retry with each track collapsed to its best hypothesis, and if that fails treat the look as missed; count every fallback

First result. On the first 60 s of S12 seed 20, MFA covered objects exactly as well as the GNN (0.844) with no false-track rows, but confirmed 81 tracks where the GNN confirmed 33, and took 92 s of tracker time, 93% of it inside Stone Soup's associator (74 ms per call, largest cluster 20 tracks). The extra tracks are the first thing to investigate.

Lesson for integration. A research library that works on its own examples needs guard rails before it runs unattended over a dense scenario: every failure must be caught, handled conservatively and counted, so that a rare numerical edge case costs one look's association rather than a whole run. The MFA results on S12 seeds 18–20 are in progress and will be added here.

11 The stand-ins revisited

Table 13 updates Part 1's ranked list of stand-ins (Part 1, Table 14). The largest departure from the real world is gone, and two others are reduced. The results in this part are therefore closer to what a fielded chain would see, with two caveats that remain: false tracks are formed but not scored, and the filter has no maneuver model, which matters most for Tier 3 in the terminal phase.

Table 13. Part 1's stand-ins with a real tracker.

Part 1 rank	Stand-in	Status with a real tracker
1	Association by label	Replaced. Five trackers; label mode kept as the reference
3	False alarms dropped before tracking	Partly replaced. Clutter reaches the tracker and false tracks form (about 4% of track-snapshots), but false-track rows are not yet scored by CID or TP, so a false track a real system might engage is not counted
5	One constant-velocity filter per labeled track	Partly replaced. Tracks are formed and broken realistically and registration bias is allowed for, but each track still uses one constant-velocity filter (no maneuver model), and Doppler is used only for the first tier guess, not in gating or updates
8	Tracker settings chosen by true tier	Replaced in tracker modes: settings follow the observed tier
2, 4, 6, 7, 9	Same-simulator training and test; all Tier 3 hostile; simplified sensors; profile replay; graph gate by true tier	Unchanged

12 Conclusions and next steps

What the tracker work shows:

- **A real tracker costs the threat list about 4 points, not the identification models.** With perfect identity, top-3 capture falls from 88.9% to 85.2% behind the GNN. CID balanced accuracy moves by under a point per tier, and the stack still matches perfect identity in the ranking with no friends in ENGAGE.
- **The loss is concentrated in groups the radar cannot resolve,** chiefly the hostile drone swarm. Almost every threat is still covered by some track (97.8%); what is lost is each member having its own track. This is a sensor-resolution limit that no association method fully removes.
- **Scoring had to change with the tracker.** Measured only over tracks in the picture, the GNN looked better than label mode. Scoring against all threats from first detection is now the standard and should be used for every tracker comparison.
- **Protection of resolved tracks is the most valuable single change.** It is free and works with every tracker. With it, soft association helps in swarms (VB about 10 points over the GNN on the bench), but on S12 VB with protection beats the GNN with protection by only 0.5 points of own-track share, at 10–45 times the run time.
- **Better association alone did not change Tier 3,** whose mean coverage on the test seeds (0.90) is the same for every tracker. Tier 3 improvement must come from track start and filtering, not assignment.
- **Group counts are information, not a weight.** The RCS and nearby-echo estimator detects groups well (AUC about 0.85), but weighting threat by members hurts per-object capture.

Recommended working tracker: the GNN with protection, for its accuracy, speed and simplicity, with VB with protection (one iteration) kept for scenarios dominated by close groups and Stone Soup MFA under evaluation. Table 14 lists the next steps.

Table 14. Recommended next steps.

Priority	Next step	What it would settle
1	Finish the MFA runs on S12 seeds 18–20 and its bench cases; find why it confirms more tracks than the GNN	Whether multi-frame assignment improves on the GNN with protection, and at what cost
2	Re-run the GNN with protection, VB with protection (1 iteration) and MFA on the current S12, which now includes the team's ABM and MARV classes (Y and Z)	One like-for-like comparison of the trackers on the current scenario
3	Keep false-track rows in the threat picture and score them	Whether false tracks reach the top of the threat list or ENGAGE
4	Faster Tier 3 track start: a confirmation rule for fast, high reports, and radial speed in the gate	The Tier 3 coverage loss at track start, which no association method changed
5	Run S12F_gnn and S12D_gnn (ready), and test label-trained models on tracker output	Whether Part 1's robustness findings hold behind a real tracker, and how much retraining on tracker output matters
6	Use the group-count estimate as a CID feature or operator tag, not as a threat weight	Whether group awareness helps identification or the operator without distorting the ranking
7	A maneuver model (IMM) for Tier 3 tracks	Track continuity in the weave and the terminal dive
8	A technical note on the trackers' algorithms, their similarities and differences	A reference for choosing and tuning trackers